

Contribution Guideline

Overdue Task Prediction -- PMS Project

Branching | Pull Requests | Merging
Read this before making any commits or PRs.

1. Branch Types

1.1 Standard Branches (Assigned to a Phase)

You work here if you have been assigned to this phase. These are the main development branches where all feature work happens.

Branch	Phase	Who Works Here?	Merges To
dev-data	Phase 1 (Data & Infra)	Team A, B, C (assigned)	staging
dev-eda-modeling	Phase 2 (EDA & Modeling)	Team D, E (assigned) + F joins EDA	staging

1.2 Non-Standard Branches (Not Assigned)

Use this if you were NOT assigned to a phase but still want to contribute. Create your own branch and send a PR to the standard branch.

```
dev-<standard-branch>-<your-first-name>

Examples:
dev-data-abebe      # Abebe (not assigned) wants to help dev-data
dev-eda-modeling-helen # Helen (not assigned) wants to help eda-modeling
dev-data-john       # John (not assigned) wants to help dev-data
```

1.3 Protected Branches

Branch	Purpose	Protected?	Source	Merge To
staging	Integration. PRs from dev-* merge here.	YES (1 approval)	dev-*	main
main	Production. Tagged releases.	YES (2 approvals)	staging	--

1.4 Branch Flow Diagram



2. Workflow

2.1 I Am Assigned to This Phase (Standard Branch)

You are on Team A, B, or C for Phase 1 (dev-data) or Team D, E for Phase 2 (dev-eda-modeling). You work directly on the standard branch.

```
# Start: checkout the standard branch
git checkout dev-data

# Always pull latest staging first
git pull origin staging

# Work, commit, push
git add .
git commit -m "[dev-data] add analytical dataset SQL"
git push origin dev-data

# Open PR: dev-data -> staging
```

2.2 I Am Not Assigned (Non-Standard Branch)

You want to contribute but are not on the phase team. Create your own branch and PR to the standard branch.

```
# Start: checkout the standard branch
git checkout dev-data

# Create your personal contributor branch
git checkout -b dev-data-abebe

# Pull latest from standard branch
git pull origin dev-data

# Work, commit, push
git add .
git commit -m "[dev-data-abebe] add data quality checks"
git push origin dev-data-abebe

# Open PR: dev-data-abebe -> dev-data
```

3. Pull Request Process

3.1 Before Opening a PR -- Resolve Conflicts Locally

This is the most important rule. BEFORE you open a PR, pull the target branch, merge it into your branch, and resolve any conflicts on your own machine. This keeps the remote branch clean.

Example: dev-data-abebe wants to PR to dev-data

```
git checkout dev-data-abebe
git pull origin dev-data      # get latest from target
git merge dev-data            # merge target into your branch
# If conflicts appear, fix them now in your editor
# git status    (see which files conflict)
# fix files, then:
# git add <fixed-files>
# git commit -m "resolve merge conflicts"
git push origin dev-data-abebe # push the clean branch
# NOW open the PR (no conflicts shown)
```

This applies to ALL PRs:

- Non-standard -> Standard: pull standard branch, merge, push, PR
- Standard -> Staging: pull staging, merge, push, PR
- Staging -> Main: pull main, merge, push, PR

3.2 PR Requirements

- Title: [<branch-name>]: <what changed>
- Description: what was done + files changed + how to test
- 1 reviewer from a different team
- All tests must pass
- No merge conflicts (resolved locally before push)

3.3 Review Checklist

- SQL: NULL handling? DISTINCT ON for M:M joins? COALESCE for dept?
- Modeling: time-based split? No target leakage? shift(1) on lag features?
- Tests: exist? passing?
- Docs: handoff docs updated for next phase?
- No secrets, absolute paths, or local machine references

4. Merge Strategy

4.1 Merge Order

Non-standard -> Standard	(dev-data-abebe -> dev-data)
Standard -> Staging	(dev-data -> staging)
Staging -> Main	(staging -> main, tag v1.0)

4.2 Merge Rules

- Non-standard -> Standard: merge. Branch stays (we keep all branches).
- Standard -> Staging: squash and merge.
- Staging -> Main: merge commit. Tag as v1.0.
- NO direct commits to staging or main.
- Staging -> Main requires 2 approvals from different teams.

4.3 Conflict Resolution

```
git checkout dev-data-abebe
git fetch origin dev-data
git merge origin/dev-data
# resolve conflicts in editor
git add <resolved-files>
git commit -m "resolve merge conflicts"
git push origin dev-data-abebe
```

5. Quick Reference Card

ASSIGNED? --> Work directly on: dev-data or dev-eda-modeling
PR to staging

NOT ASSIGNED? --> Create: dev-<standard>-<your-name>
Example: dev-data-abebe
PR to the standard branch (e.g. dev-data)

BEFORE EVERY PR:

1. git checkout <your-branch>
2. git pull origin <target-branch>
3. git merge <target-branch>
4. Resolve conflicts locally
5. git push origin <your-branch>
6. Open PR (clean, no conflicts)

MERGE PATH:

dev-data-abebe -> dev-data -> staging -> main (v1.0)

REVIEWS:

dev-* -> Staging: 1 approval (different team)
Staging -> Main: 2 approvals (different teams)